RESEARCH PERSPECTIVE | OPEN INVITATION TO COLLABORATE

Reliable Mathematics for Quantum Research

What new scientific opportunities arise when mathematical models used in quantum experiments can be evaluated repeatedly, efficiently, and with transparent numerical quality?

Progress in quantum research depends on more than improving individual quantum devices. Classical computations also play a central role in calibrating experiments, interpreting measurement data, and studying theoretical models. An especially promising setting is one in which the same mathematical function must be evaluated for many different inputs. A reusable mathematical execution engine may offer an additional computational component for such work. paceval UG invites research partners to investigate whether it can reduce repeated computational effort, support consistent reuse of mathematical models, and provide useful numerical checks. [1, 4, 18]

What is paceval?

paceval is a mathematical runtime developed by paceval UG. It converts a text-based mathematical expression into a reusable execution structure. Through a documented application programming interface for C and C++, that structure can be evaluated with changing inputs, including batches of input values. The interface provides persistent computation objects. paceval also offers numerical interval information through a feature called Trusted Interval Computing (TINC). These capabilities motivate two related research questions. How efficient is repeated execution in a quantum research workflow? How informative are the reported numerical intervals for the mathematical models under study? [1, 2, 3]

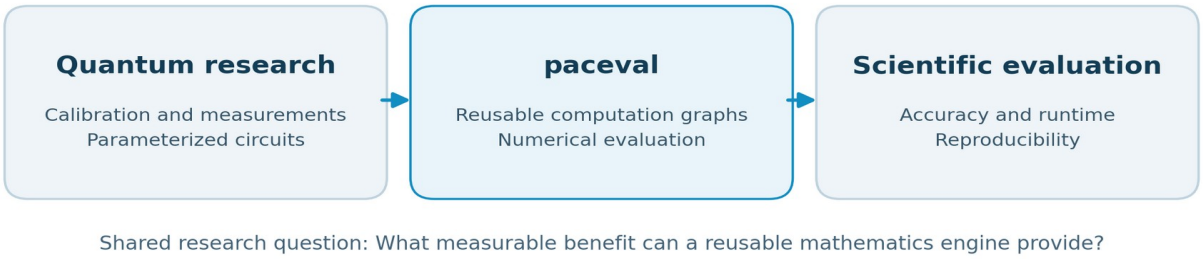


Figure 1. Proposed integration of a reusable mathematics engine into established research workflows. The diagram describes an evaluation concept, not an existing deployment on a quantum computer.

Our offer. paceval UG is willing to make its mathematical engine available for jointly defined research projects. We can support technical integration and are open to developing additional application programming interface functions together with researchers whenever these are scientifically useful.

Purpose of this white paper

We propose an independently reproducible study of whether the current paceval capabilities can add value to well-defined classical tasks in quantum research. Two initial demonstrations are envisaged. The first concerns repeated evaluation of mathematical models used for calibration and measurement analysis. The second concerns execution of explicit mathematical functions derived from suitable parameterized quantum circuits. Numerical accuracy, computational effort, and integration into existing research software would be assessed together.

1 The opportunity for quantum research

The proposed research opportunity is to make explicitly defined mathematical functions available as standalone, reusable computation objects. The same object could support multiple experiments, optimization routines, or analysis programs without reimplementing the mathematical expression for each use. The relevant scientific outcomes would be lower computational cost for repeated evaluations, consistent model definitions across experiments, and more transparent numerical quality checks. Reproducible tests can establish which, if any, of these benefits materialize for particular workloads. [1, 2]

This study would build upon established quantum research software. Qiskit Experiments supports device characterization and calibration, while QuTiP and its control package support quantum dynamics and control. paceval could take responsibility for a specifically defined part of the classical mathematics without replacing the underlying physical model. The same scientific computation could then be evaluated using the incumbent implementation and a paceval module. That approach allows any additional mathematical execution or validation capability to be assessed in its real research context. [4, 5, 6]

Research task	Specific scientific opportunity
Repeated calibration and sensor models	Prepare a fixed mathematical expression once and evaluate it across many measurements, delays, and parameter settings.
Parameter studies and classical optimization	Share a precisely defined mathematical computation object across experiments and optimization programs.
Numerically sensitive calculations	Check values and TINC interval outputs against independently computed references.
Integration with existing tools	Test an isolated mathematical subproblem through an interface without altering the rest of the scientific software.

Three sources of uncertainty must remain distinct. Statistical uncertainty arises from finite measurement samples. Model uncertainty concerns whether a mathematical description adequately represents the physical system. Numerical uncertainty arises from classical computations, including rounding and conditioning effects. The proposed paceval experiments address repeated evaluation of explicit models and numerical checks on those evaluations. Separating these sources enables researchers to identify which part of a result is affected by the computation method. [12, 13, 16]

2 Research direction A: calibration and mathematical analysis

A suitable first study is the evaluation of a Ramsey experiment. A qubit is subjected to two appropriately configured control pulses separated by a variable delay. The resulting oscillation can be used to estimate the detuning frequency and an effective dephasing time. A common phenomenological model with exponential decay gives the probability of a selected measurement outcome as follows. [5]

$$p(t;\theta) = \tfrac{1}{2}\left[1 + C \cdot \exp(-t/\tau) \cdot \cos(2\pi\,\Delta f \cdot t + \varphi)\right]$$

Here t is the delay time, the contrast C lies between zero and one, and τ is a positive effective decay time. The parameter Δf denotes frequency detuning, while φ is the phase. For this pilot study, the model is intentionally simple and serves as a reproducible mathematical benchmark. Parameter definitions and physical units would remain identical across all implementations.

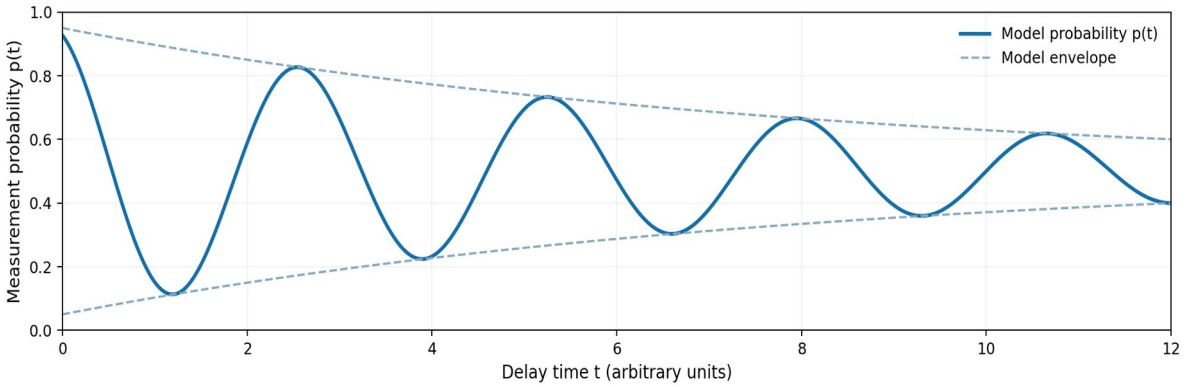


Figure 2. Illustrative Ramsey probability with exponentially damped oscillations and model envelopes. The curves use synthetic parameters (C = 0.90, τ = 8, Δf = 0.37, φ = 0.32, in consistent arbitrary time units). No experimental measurements are shown.

If k_i events are observed among N_i independent repetitions at each delay, the observations can be modeled conditionally on fixed probabilities by a binomial distribution. Ignoring terms independent of the model parameters, the negative log-likelihood is:

$$L(\theta) = -\Sigma_i \left[k_i \ln p_i + (N_i - k_i) \ln(1 - p_i) \right]$$

The study would use identical input data, parameter bounds, initial estimates, statistical objectives, and stopping criteria. Boundary cases with p = 0 or p = 1 would be handled using the mathematical limits of the binomial log-likelihood, rather than arbitrary clipping without explanation. We would measure the time needed to create computation objects, the cost per evaluation, and the time required for a complete parameter fit. These results would show whether a possible gain in the mathematical kernel improves total analysis time as the number of data sets grows.

3 Research direction B: explicit mathematical quantum models

A second line of research extends beyond calibration formulas. For suitable circuits built from parameterized gates with fixed finite spectra, expectation values can be expressed as finite Fourier series in the circuit parameters. This mathematical result is established in peer-reviewed research. PennyLane already supports Fourier analysis and reconstruction of selected parameter dependencies. We propose evaluating an explicit function obtained by such methods as a reusable paceval computation over many parameter sets. This may provide a portable mathematical execution layer for parameter sweeps, optimization studies, and numerical cross-checks. [7, 8, 9, 10]

$$f(\theta) = \langle 0|U(\theta)^\dagger \, O \, U(\theta)|0\rangle = \Sigma_{\omega \in \Omega} c_\omega \exp(i \, \omega \cdot \theta)$$

This method is most promising when the explicit function is reasonably compact and evaluated many times. Constructing or estimating its Fourier coefficients must therefore be counted as a separate cost. The number of terms can grow rapidly with model complexity, and characterizing that scaling is part of the proposed research. The objective is to make a useful mathematical representation reusable for suitable models. It is not to replace general quantum computations with an efficient classical expression.

An analytically verifiable two-qubit example

We initialize two qubits in the state $|00\rangle$. Each qubit undergoes a rotation about the y axis with its own angle, θ_1 or θ_2. A controlled-NOT gate then uses the first qubit as the control and the second as the target. For the observable $I \otimes Z$, the exact expectation value is:

$$f(\theta_1, \theta_2) = \cos\theta_1 \cdot \cos\theta_2 = \tfrac{1}{2}[\cos(\theta_1 - \theta_2) + \cos(\theta_1 + \theta_2)]$$

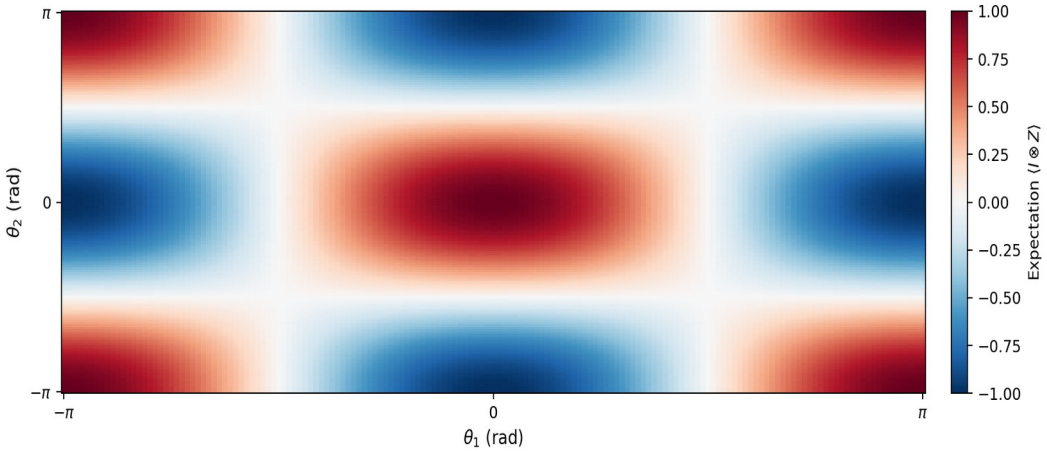


Figure 3. Exact expectation value of I ⊗ Z for the specified two-qubit circuit. The plot is generated from the analytical formula and does not contain quantum processor measurements.

The pilot would begin with this analytically verifiable circuit, then progressively increase parameter count and Fourier complexity. For each workload, we would determine how many evaluations are needed to amortize one-time preparation costs. If Fourier coefficients require quantum measurements, their acquisition cost and statistical uncertainty must be included. This would identify the model families for which execution with paceval has practical value.

Extension to quantum sensing and quantum metrology

Quantum sensing and quantum metrology are further potential application areas. Physical parameters are inferred from quantum measurement signals, and calibration or sensor response models are often evaluated repeatedly. A suitable collaborative study could select a concrete signal, a specified estimation model, and a reproducible set of numerical evaluations. It would ask whether paceval can make the classical mathematical analysis or its numerical checking more efficient or easier to integrate without changing the scientific meaning of the result. [17, 18]

4 Numerical quality and scientific reproducibility

Another promising direction is to combine reusable execution with independent numerical checking. According to its public documentation, paceval provides interval information through Trusted Interval Computing (TINC). We propose testing these outputs against independent references for selected expressions. GNU MPFR supports high-precision arithmetic with specified rounding behavior. FLINT and its Arb components implement rigorous ball arithmetic for enclosing numerical results. The scientific question is whether paceval provides interval information that is sufficiently reliable, informative, and efficient for practical calibration and model calculations. [3, 12, 13]

Evaluation criterion	Scientific assessment
Numerical accuracy	Deviation from an independent, higher-precision reference, including ill-conditioned or limiting cases.
Quality of interval outputs	Compare TINC outputs with exact test values where available and with rigorous FLINT/Arb reference enclosures. Report containment tests and interval widths.
Reproducibility	Repeated execution with fully recorded inputs, library versions, platforms, and arithmetic settings.
Performance and scaling	Creation cost, amortized evaluation time, throughput, memory use, and total analysis time.
Scientific integration	Interface complexity, required extensions, and impact on an established analysis workflow.

A useful outcome for quantum researchers could be the ability to report a numerical result alongside a separately evaluated statement about its computational accuracy. The study would compare calculated values with independently checked numerical results, including difficult parameter regimes. Statistical uncertainties in measured data or estimated Fourier coefficients would be assessed separately from floating-point errors. This distinction can matter when signals are weak, parameter scales differ markedly, or two computed values are nearly equal. [12, 13, 16]

5 How paceval could complement existing research software

The practical value of a mathematical engine depends on its ability to handle defined subproblems within established scientific workflows. paceval already provides a C and C++ interface suitable for exploring such integration. An adapter could, for example, pass a calibration objective or explicit Fourier expression to a persistent paceval computation object. OpenQASM 3.1 also specifies calls to external classical functions. This provides a possible integration concept, although actual support and latency depend on the implementation and its target hardware. [2, 14]

Research workflow	Potential contribution of a paceval module
Repeated qubit calibration	Evaluate the same parameterized mathematical model for many new measurements.
Control parameter optimization	Supply a precisely specified objective function or constraint as an interchangeable component.
Fourier models of quantum circuits	Evaluate an explicitly reconstructed mathematical function repeatedly for new parameters.
Numerical quality assurance	Investigate additional checks on numerical results and interval outputs for selected models.
CPU and hardware-oriented systems	Test integration through C/C++ in software first, then explore hardware when there is a suitable workload.

Quantum research already benefits from mature software ecosystems. Qiskit Experiments supports calibration workflows, QuTiP supports quantum control, PennyLane offers Fourier analysis tools, JAX supports compilation and reuse of numerical functions, and Quanlse provides quantum control capabilities. Their existence makes it possible to test paceval as an interchangeable mathematics component while retaining established physical models and research workflows. A meaningful study would compare the total computation with equivalent numerical requirements and would include preparation, data movement, and integration costs in addition to kernel execution time. [4, 6, 9, 11, 15]

Outlook for hardware acceleration

If a pilot project identifies computationally substantial expressions that are evaluated repeatedly, a further opportunity may lie in dedicated hardware. Their execution could be investigated on a field-programmable gate array (FPGA) and, in a later phase, in specialized hardware. The relevant performance measures would be actual end-to-end latency, throughput, energy consumption, and data transfer. The current paceval application programming interface is a starting point for software integration. Any hardware-based quantum research application would require separate design and experimental validation.

6 Proposal for an open collaborative study

Phase	Collaborative activity	Expected scientific outcome
1. Select a task	Researchers identify a repeatedly evaluated computation, input data, and numerical requirements.	A falsifiable hypothesis about practical value.
2. Build the pilot	Implement the same mathematical function in paceval and an established baseline.	Comparison without changing the underlying physical problem.
3. Validate numerics	Independently check values, difficult boundary cases, and interval outputs.	A documented profile of numerical reliability.
4. Evaluate in context	Measure preparation, repeated execution, full workflow time, and integration effort.	Evidence about whether broader scientific integration is worthwhile.

An open research invitation from paceval UG

paceval UG would like to contribute its mathematical engine as an additional research tool for quantum science. We invite universities, research institutes, and industrial research groups to identify a scientifically relevant and mathematically precise subproblem that can be evaluated with paceval. We are happy to support integration and to develop missing application programming interface capabilities or numerical interfaces jointly when these would benefit the research.

A collaboration can begin with a small, independently reproducible reference case and then expand to real measurement data or more complex models. We would seek to document methods, comparison conditions, and outcomes transparently with our partners. Agreements on software access, joint development, and publication of results would be established collaboratively in advance.

Terminology and references

Brief terminology guide

An application programming interface (API) is a software interface through which another program invokes a library. An execution graph is an internal structure for repeatedly evaluating a mathematical expression. Trusted Interval Computing (TINC) is the name used by paceval for its numerical interval information. A qubit is a quantum system used to represent quantum information. A field-programmable gate array (FPGA) is a reconfigurable hardware device. Just-in-time compilation (JIT) generates optimized executable code at runtime for subsequent reuse. A Fourier representation expresses a function as a sum of trigonometric or complex exponential terms.

Scientific and technical references

[01] paceval UG. Mathematical Engine. Overview of mathematical expression processing and reusable execution structures. Vendor documentation. Open source

[02] paceval UG. Application Programming Interface, version 4.25. Computation objects and software interface. Vendor documentation. Open source

[03] paceval UG. TINC: Trusted Interval Computing. Vendor description of numerical interval information. Open source

[04] Kanazawa et al. (2023). Qiskit Experiments: A Python package to characterize and calibrate quantum computers. Journal of Open Source Software 8(84), 5329. Open source

[05] Qiskit Experiments. T2* Ramsey Characterization. Technical documentation of the Ramsey calibration model. Open source

[06] QuTiP. qutip-qtrl: Quantum Optimal Control. Documentation of the GRAPE and CRAB methods. Open source

[07] Schuld, Sweke and Meyer (2021). Effect of data encoding on the expressive power of variational quantum-machine-learning models. Physical Review A 103, 032430. Open source

[08] Wierichs et al. (2022). General parameter-shift rules for quantum gradients. Quantum 6, 677. Open source

[09] PennyLane. Fourier coefficients. Technical documentation. Open source

[10] PennyLane. Fourier reconstruction. Technical documentation. Reconstruction is explicitly documented for individual parameter directions. Open source

[11] JAX. Just-in-time compilation. Official documentation of compilation and caching. Open source

[12] GNU MPFR Project. The GNU MPFR Library. High precision floating-point arithmetic with specified rounding. Open source

[13] FLINT. Ball arithmetic using Arb. Documentation of rigorously enclosing arithmetic. Open source

[14] OpenQASM 3.1. Classical instructions. Specification of calls to external classical functions. Open source

[15] Baidu Research. Quanlse. Open-source software for quantum control and calibration. Open source

[16] Cai et al. (2023). Quantum error mitigation. Reviews of Modern Physics 95, 045005. Open source

[17] Budker, Shaffer and Kitching (2025). Atom-Based Quantum Sensing of Electromagnetic Fields. Optica 12(12). DOI: 10.1364/OPTICA.569334. Open source

[18] Marciniak et al. (2022). Optimal metrology with programmable quantum sensors. Nature 603, 604 to 609. Open source

References checked on 4 November 2026. Descriptions of paceval functionality are based on public vendor documentation. The proposed study would independently assess performance and numerical properties for the selected quantum research applications.

paceval UG (limited liability company) | Munich, Germany | paceval.com | info@paceval.com